

NETWORK INFRASTRUCTURE MANAGEMENT SYSTEM

Full-Stack Web Development Semester Project

Project Description Document

Technology Stack: React | Java | Spring Boot | REST API | Hibernate/JPA | Maven | PostgreSQL

Working Name: NetManage

1. Project Overview

Network Infrastructure Management System is a web-based CRUD application for recording and managing the network infrastructure of an organization. The system provides a centralized interface for maintaining structured records about network devices, device types, locations, interfaces, IP addresses, subnets, VLANs, users, and maintenance activities.

The project is intentionally limited to Create, Read, Update, and Delete (CRUD) functionality. Its purpose is to demonstrate sound relational database design, object-relational mapping, RESTful backend development, and frontend integration using the prescribed technology stack. Advanced network automation, real-time monitoring, routing analysis, and other non-CRUD functionality are outside the project scope.

2. Problem Statement

Network infrastructure information is often recorded across spreadsheets, paper records, configuration notes, and disconnected documents. This makes it difficult to maintain consistent records, identify where equipment is located, track interfaces and IP addresses, and maintain a reliable history of maintenance activities.

The proposed system addresses this problem by providing one structured web application in which authorized users can create, view, update, and delete infrastructure records. Related information is stored in a PostgreSQL relational database and exposed through a Spring Boot REST API to a React frontend.

3. General Objective

To design and implement a web-based Network Infrastructure Management System that provides centralized CRUD management of organizational networking assets and related infrastructure records using React, Java, Spring Boot, REST API, Hibernate/JPA, Maven, and PostgreSQL.

4. Specific Objectives

- Design a normalized relational database for storing network infrastructure information.
- Implement persistence and entity relationships using Hibernate/JPA.
- Develop RESTful CRUD endpoints using Spring Boot and Java.

- Develop a React frontend for managing and displaying infrastructure records.
- Implement validation and referential integrity for related entities.
- Provide organized management of network devices, interfaces, IP addresses, VLANs, subnets, locations, users, and maintenance records.
- Demonstrate a complete full-stack workflow from PostgreSQL database to backend API to frontend interface.

5. Project Scope

The application will provide CRUD functionality for the following entities:

Entity	Purpose
User	Stores system users and their roles.
Device Type	Stores categories such as Router, Switch, Firewall, Access Point, and Server.
Location	Stores the physical location of network equipment.
Device	Stores network equipment details such as hostname, manufacturer, model, serial number, status, and purchase date.
Interface	Stores interfaces belonging to network devices, including interface name, type, MAC address, status, and VLAN association.
VLAN	Stores VLAN numbers, names, descriptions, and status.
Subnet	Stores network address, CIDR, subnet mask, description, and VLAN association.
IP Address	Stores IP address records and their association with interfaces and subnets.
Maintenance Record	Stores maintenance activities associated with devices and users.

6. Functional Requirements

ID	Module	Requirement
FR-01	User Management	Create, view, edit, and delete user records.
FR-02	Device Type Management	Create, view, edit, and delete device types.
FR-03	Location Management	Create, view, edit, and delete physical locations.
FR-04	Device Management	Create, view, edit, and delete network device records.
FR-05	Interface Management	Create, view, edit, and delete device interface records.
FR-06	VLAN Management	Create, view, edit, and delete VLAN records.
FR-07	Subnet Management	Create, view, edit, and delete subnet records.
FR-08	IP Address Management	Create, view, edit, and delete IP address records.
FR-09	Maintenance Management	Create, view, edit, and delete maintenance records.

FR-10	Relationship Management	Allow related entities to be selected through foreign-key relationships in CRUD forms.
-------	-------------------------	--

7. Core Database Relationships

Parent Entity	Child Entity	Cardinality
DeviceType	Device	1: N
Location	Device	1: N
Device	Interface	1: N
VLAN	Interface	1: N
VLAN	Subnet	1: N
Subnet	IPAddress	1: N
Interface	IPAddress	1: 0..1
Device	MaintenanceRecord	1: N
User	MaintenanceRecord	1: N

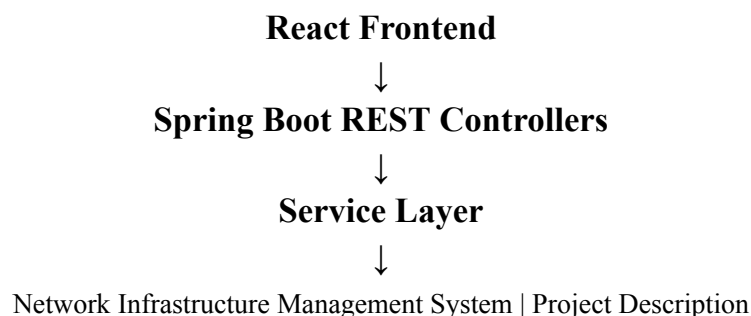
These relationships are intended to provide a realistic relational model while remaining manageable within the CRUD-only project constraint. Foreign keys will enforce referential integrity at the database level, while Hibernate/JPA will map these relationships to Java entities.

8. Technology Stack

Technology	Role in Project
React	Frontend user interface and CRUD views/forms.
Java	Primary backend programming language.
Spring Boot	Backend application framework and REST API implementation.
REST API	HTTP/JSON interface between the React frontend and backend.
Hibernate/JPA	Object-relational mapping between Java entities and PostgreSQL tables.
Maven	Project build, dependency management, and lifecycle management.
PostgreSQL	Relational database management system.

9. High-Level Architecture

The system will follow a layered architecture in which the frontend, backend, persistence layer, and database have clearly separated responsibilities.



Hibernate/JPA Repositories



PostgreSQL Database

The React frontend will communicate with the Spring Boot backend using HTTP requests and JSON responses. The backend will handle request processing and business validation, while Hibernate/JPA will manage persistence and object-relational mapping to PostgreSQL.

10. Project Modules

Module	Description
Dashboard	Provides a basic overview of stored infrastructure records and navigation to CRUD modules. No advanced monitoring or analytics are included.
Users	CRUD management of system user records.
Device Types	CRUD management of device categories.
Locations	CRUD management of physical infrastructure locations.
Devices	CRUD management of networking equipment.
Interfaces	CRUD management of device interfaces.
VLANs	CRUD management of VLAN records.
Subnets	CRUD management of subnet records.
IP Addresses	CRUD management of IP address records.
Maintenance	CRUD management of device maintenance records.

11. CRUD Operation Model

Every principal module will follow the same basic CRUD pattern:

Operation	Implementation
Create	A form is used to submit a new record to the REST API.
Read	The frontend retrieves and displays existing records, including relevant related fields.
Update	An existing record is loaded into an editable form and submitted for modification.
Delete	A selected record is removed subject to the database foreign-key constraints.

12. Scope Exclusions

To comply with the CRUD-only project requirement and keep the semester scope controlled, the following are explicitly excluded from the initial project:

- Real-time network monitoring or alerting.
- SNMP-based polling or device telemetry.
- Automatic router or switch configuration.
- TCP socket-based network control inside the application.
- Automatic subnet calculation or routing algorithms.
- Network topology algorithms or path analysis.

- Real-time dashboards using WebSockets.
- Automated network discovery.
- Network automation through Ansible, Terraform, or similar tools.

Networking concepts may inform the data model and example records, but the application itself remains a CRUD information-management system.

13. Course Integration

Course	Connection to Project
Full-Stack Web Development	The project directly demonstrates database, backend, REST API, Hibernate/JPA, Maven, and React integration.
Network Programming (TCP/IP) with Java	The project provides a networking-oriented domain for applying Java and network concepts without adding non-CRUD networking functionality to the application.
Data Structures and Algorithms	The project provides a practical software-development context, while algorithmic work remains within the course unless explicitly required by the project.

14. Expected Deliverables

- PostgreSQL database schema and sample data.
- Spring Boot backend project managed with Maven.
- Hibernate/JPA entity mappings and repositories.
- RESTful CRUD endpoints for the project entities.
- React frontend with CRUD pages and forms.
- Validation and appropriate error handling.
- Database relationship documentation and ER diagram.
- Project source code and setup instructions.
- Final project report and demonstration.

15. Proposed Development Sequence

Phase	Work
Phase 1	Finalize requirements and database model.
Phase 2	Create PostgreSQL schema and seed data.
Phase 3	Create Spring Boot project with Maven and configure PostgreSQL/Hibernate.
Phase 4	Implement JPA entities, repositories, and services.
Phase 5	Implement REST CRUD controllers and validation.
Phase 6	Create React application structure and CRUD interfaces.
Phase 7	Connect React frontend to the REST API.
Phase 8	Test relationships, validation, CRUD flows, and error cases.
Phase 9	Improve usability, documentation, and final presentation.

16. Project Success Criteria

- All required entities can be created, viewed, updated, and deleted through the application.
- Relationships between entities are represented correctly in PostgreSQL and Hibernate/JPA.
- The React frontend communicates successfully with the Spring Boot REST API.
- Invalid data and invalid relationships are handled without corrupting the database.
- The application can be installed, configured, and demonstrated using the documented setup process.
- The final implementation remains within the agreed CRUD-only scope.

17. Conclusion

The Network Infrastructure Management System is designed as a focused full-stack CRUD application with a realistic networking domain. Its main academic value lies in integrating relational database design, Hibernate/JPA, RESTful Java backend development, and React frontend development into one coherent system. By deliberately excluding advanced monitoring, automation, and algorithmic functionality, the project maintains a manageable semester scope while still offering sufficient relational and technical depth to demonstrate the selected technology stack.